

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory Name: A. Priyanka Reg. No.: 24011101001	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

4. MobileNetV2 Architecture

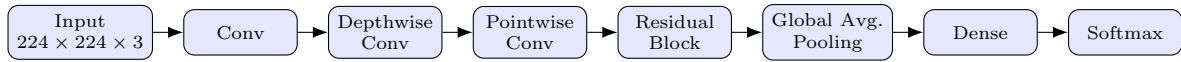
MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;

- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Source Code

The source code and implementation are available in this GitHub repository:

GitHub Repository

6. Weight Initialization

The following four initialization strategies were compared:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Plot 1: Training Loss vs. Epoch

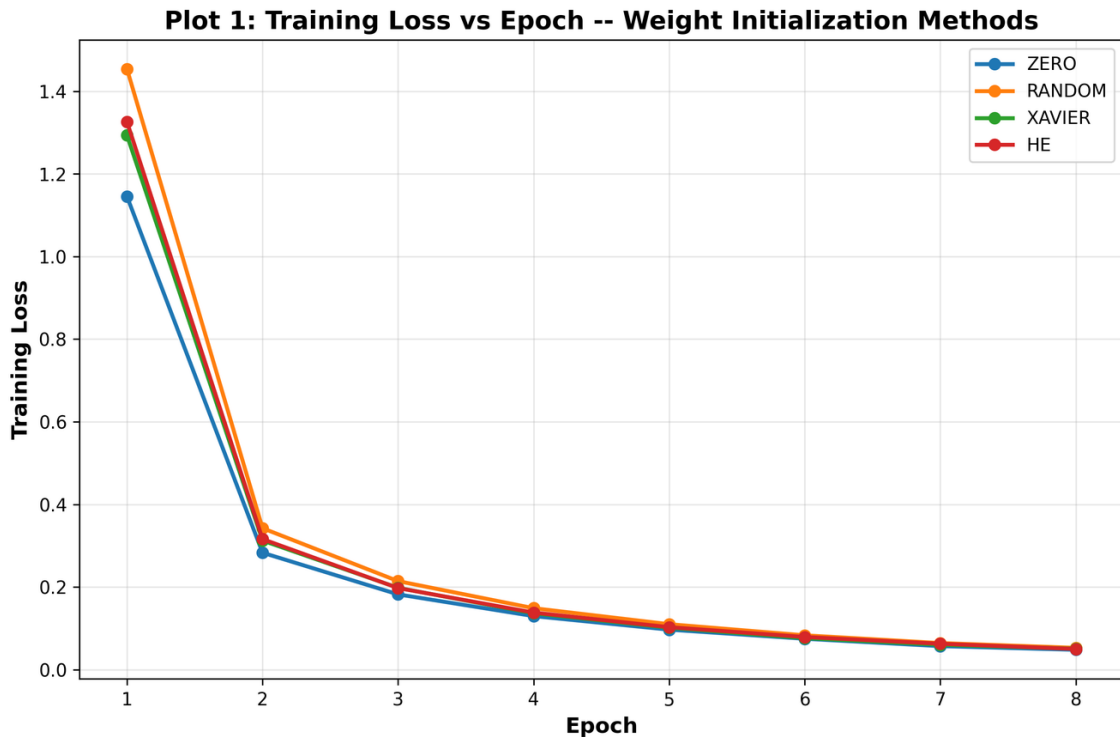


Figure 1: Training loss across epochs for different weight initialization strategies.

Inference: The plot compares the training loss for the four weight initialization methods. As the number of epochs increases, the loss decreases sharply at first and then gradually becomes lower. This means the model is learning from the training data and makes few errors. The curves become very close in the later epochs, showing that model reaches a similar level of convergence.

Plot 2: Validation Accuracy vs. Epoch

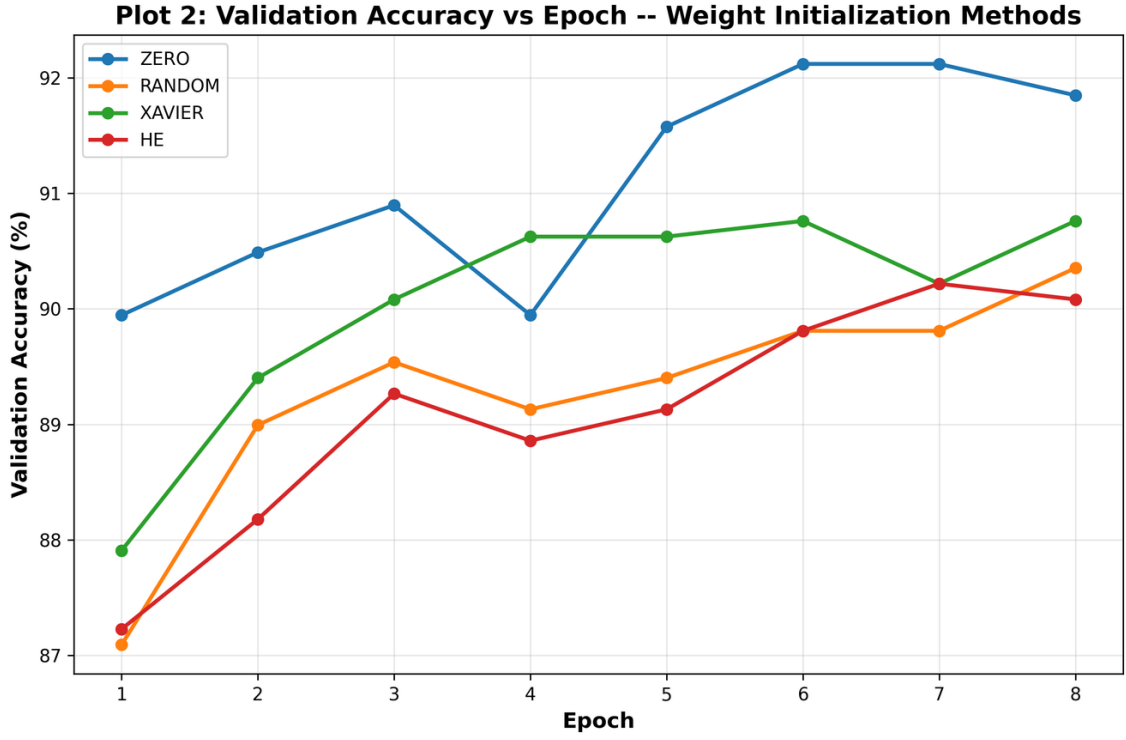


Figure 2: Validation accuracy across epochs for different weight initialization strategies.

Inference: The plot compares the validation accuracy of the four weight initialization methods. The accuracy generally increases as training goes on although some methods show small fluctuations. Zero initialization reaches the highest validation accuracy while Xavier shows a more stable improvement. This indicates that the model learns better features as training continues.

7. Regularization and Overfitting

The following approaches were compared:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Plot 3: Training and Validation Accuracy vs. Epoch

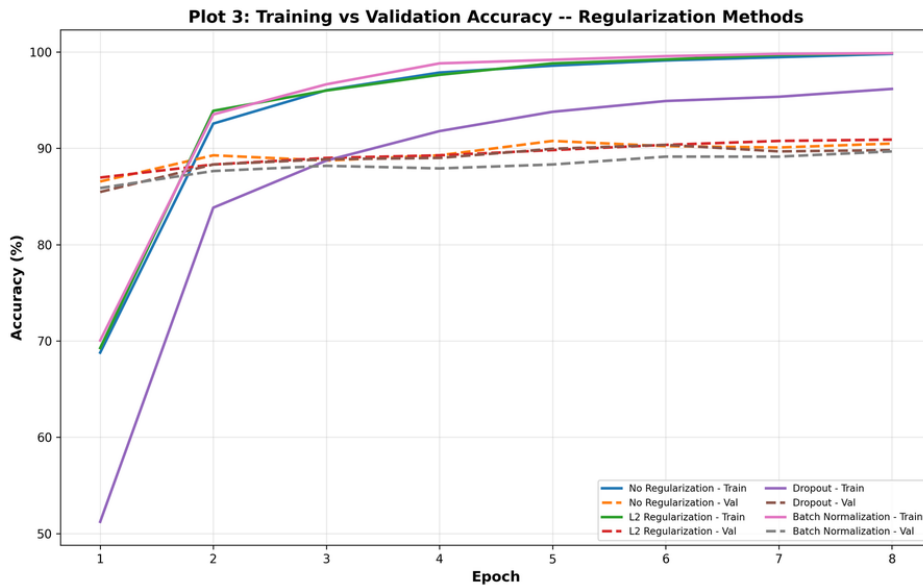


Figure 3: Training and validation accuracy for different regularization strategies.

Inference: The training and validation accuracy curves show the effect of the different regularization techniques on model generalization. The gap between training and validation accuracy indicates how well the model generalizes to unseen validation data. A smaller and more stable gap suggests better generalization and less overfitting.

Plot 4: Training and Validation Loss vs. Epoch

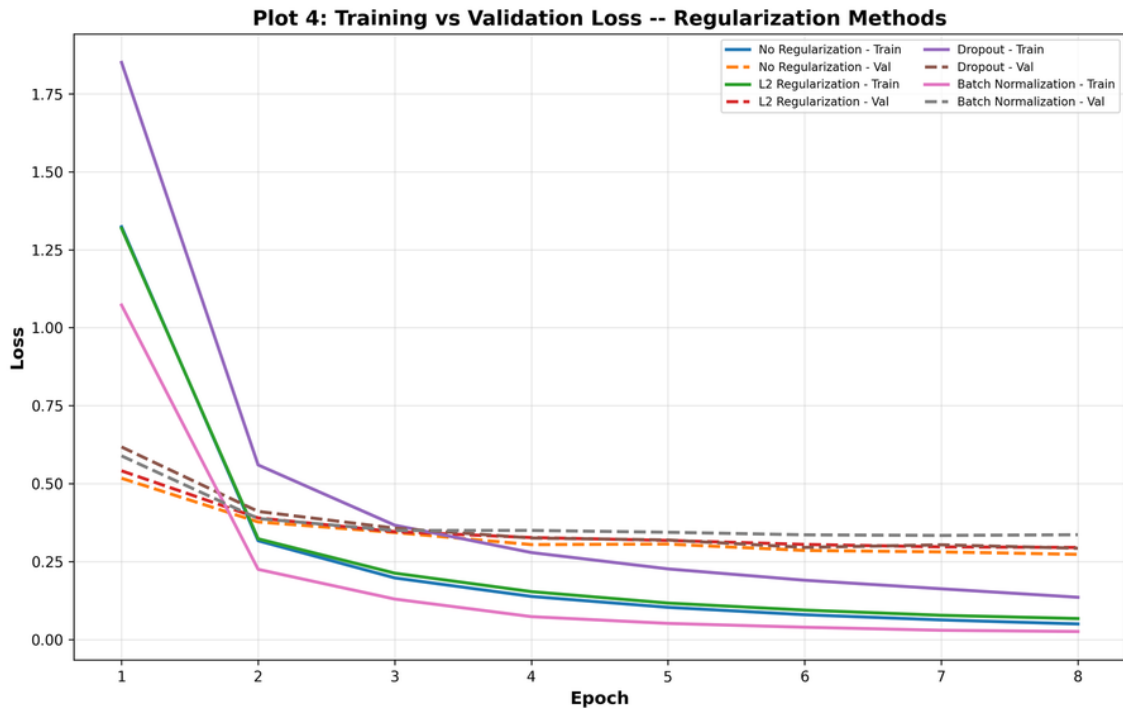


Figure 4: Training and validation loss for different regularization strategies.

Inference: Training loss keeps decreasing while validation loss stays relatively stable. The gap between the two curves shows that the model fits the training data better than the validation data and some overfitting is present.

8. Batch Normalization

Plot 5: Validation Accuracy – With vs. Without Batch Normalization

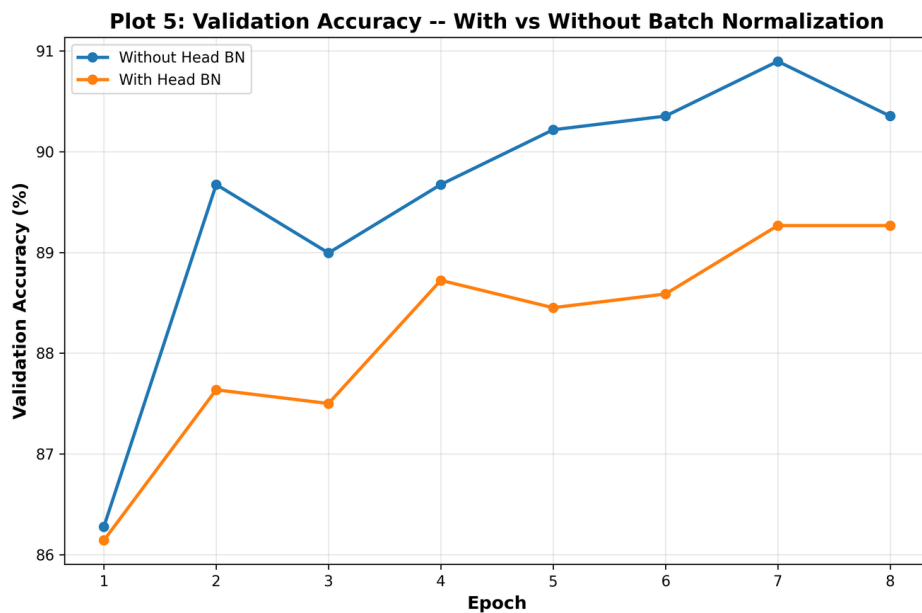


Figure 5: Validation accuracy comparison with and without Batch Normalization.

Inference: The model without BN gives higher validation accuracy across the epochs. This suggests that BN did not improve the validation performance for this experiment.

9. Optimization Algorithms

Four optimization algorithms were compared to study their convergence behaviour, stability, validation performance, and training time:

- SGD
- Momentum
- RMSProp
- Adam

Plot 6: Training Loss vs. Epoch for Different Optimizers



Figure 6: Training loss across epochs for different optimization algorithms.

Inference: The plot shows how the training loss changes for the four optimizers as the epochs increase. SGD decreases more slowly while Momentum, RMSProp and Adam reduce the loss much faster. RMSProp reaches the lowest final loss, showing faster convergence in this experiment.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

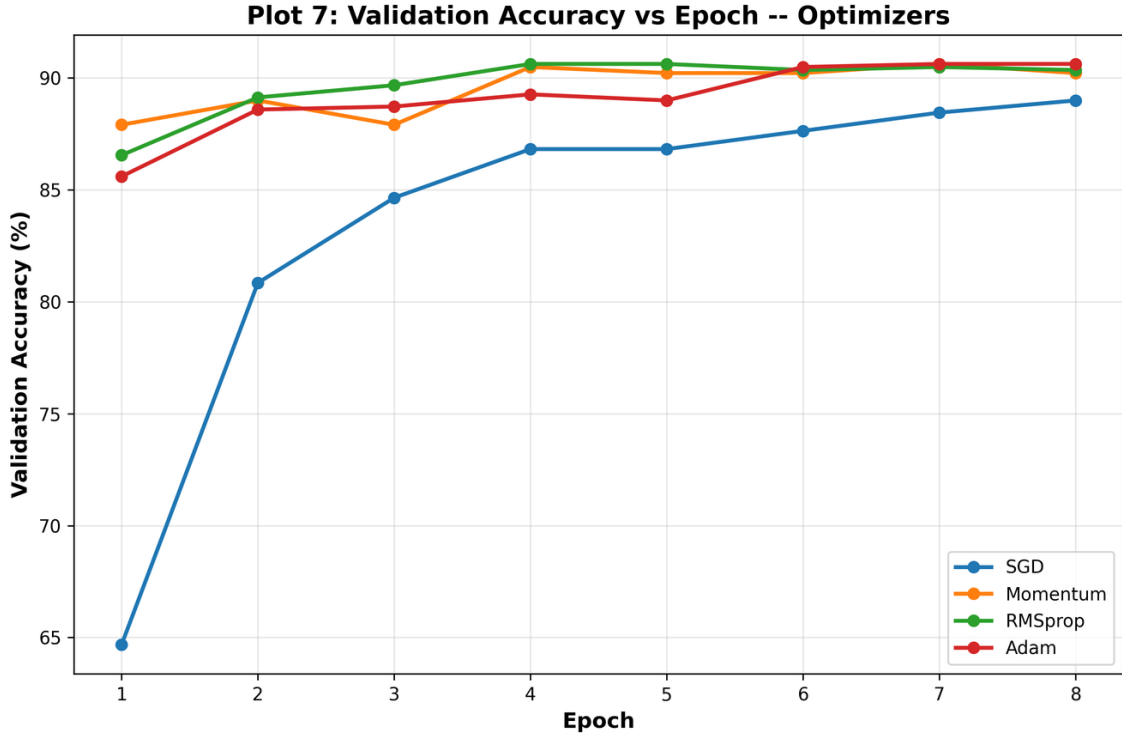


Figure 7: Validation accuracy across epochs for different optimization algorithms.

Inference: The plot compares validation accuracy for the four optimizers. SGD starts with much lower accuracy and improves gradually, while Momentum, RMSProp and Adam reach around 90% earlier. This shows that the adaptive optimizers converge faster than SGD in this experiment.

Comparison of Optimizers

Optimizer	Final Loss	Best Val. Accuracy (%)	Best Epoch	Time (s)
SGD	0.3530	88.99	8	47.93
Momentum	0.0566	90.63	7	48.55
RMSProp	0.0341	90.63	4	48.69
Adam	0.0495	90.63	7	48.20

Overall Inference: Momentum, RMSProp, and Adam achieve the same best validation accuracy of 90.63%, while RMSProp reaches this accuracy in the fewest epochs and achieves the lowest final training loss. The training times are very similar for all four optimizers, with differences of only a few seconds.

10. CNN Hyperparameter Tuning

The following hyperparameters were investigated:

- Learning Rate: 0.001, 0.0001
- Batch Size: 16, 32, 64
- Dropout Rate: 0, 0.25, 0.5
- Optimizer: SGD, Adam
- Fine-Tuning Learning Rate: 10^{-4} , 10^{-5}
- Freezing Strategy: Frozen Backbone, Partial Unfreezing

Plot 8: Learning Rate vs. Validation Accuracy

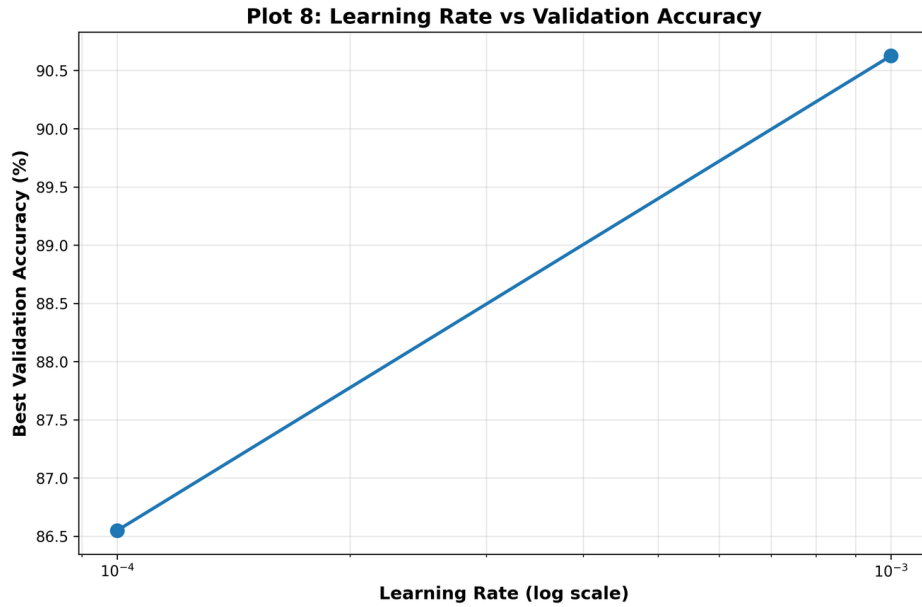


Figure 8: Effect of learning rate on best validation accuracy.

Inference: The plot shows the effect of learning rate on validation accuracy. The accuracy is higher with a learning rate of 0.001 compared to 0.0001. This suggests that the larger learning rate allowed the model to learn more effectively during the given number of epochs.

Plot 9: Batch Size vs. Validation Accuracy

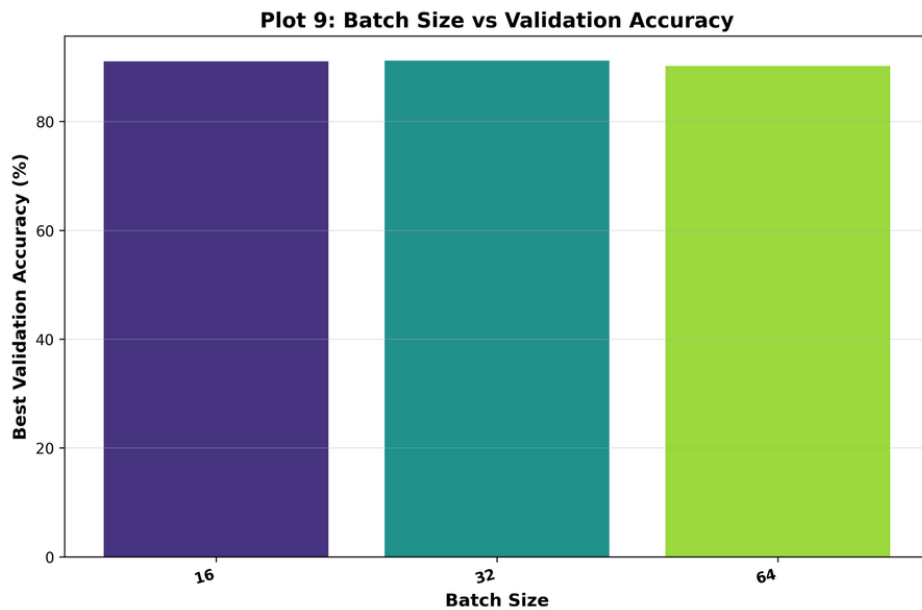


Figure 9: Effect of batch size on best validation accuracy.

Inference: In the plot, the validation accuracy of the three different batch sizes, 16, 32, and 64, are shown. There is a very minimal difference between the three cases in terms of accuracy. Therefore, the batch size has little effect on the validation accuracy.

Plot 10: Dropout Rate vs. Validation Accuracy

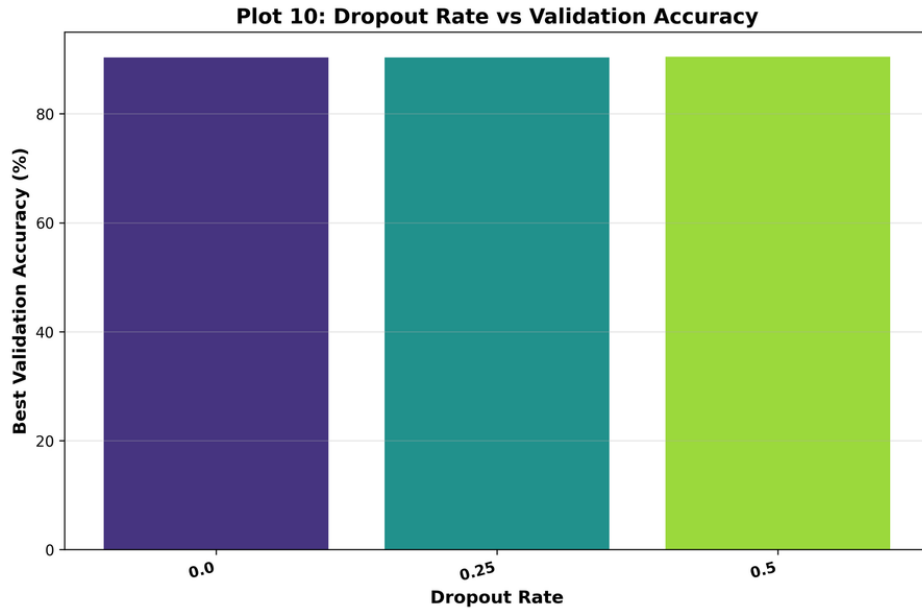


Figure 10: Effect of dropout rate on best validation accuracy.

Inference: In the graph, the validation accuracy is tested for the dropout rates of 0, 0.25, and 0.5. It can be observed that there is no significant difference between these three rates when it comes to the validation accuracy.

11. Transfer Learning and Fine-Tuning

Plot 11: Feature Extraction vs. Fine-Tuning

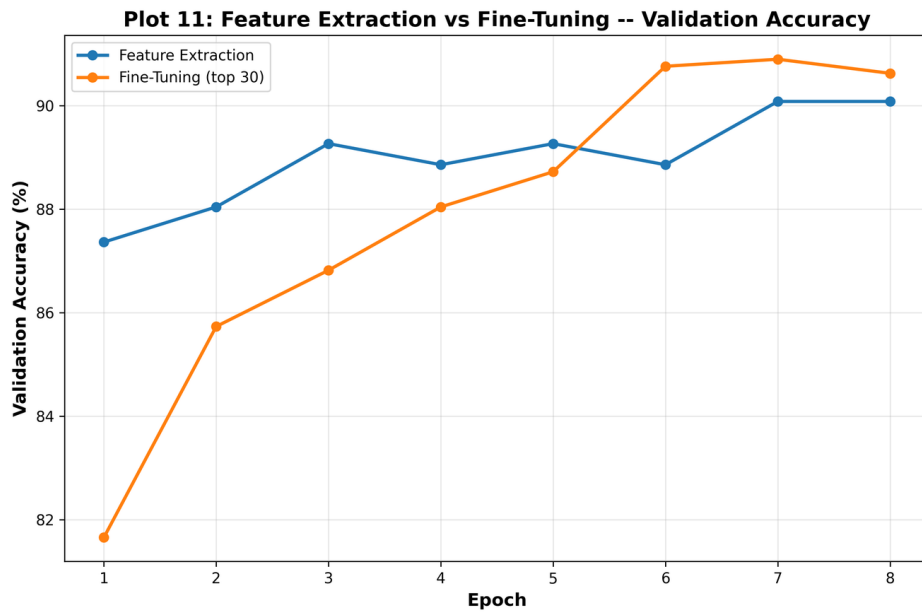


Figure 11: Validation accuracy comparison between feature extraction and fine-tuning.

Inference: The plot compares validation accuracy for feature extraction and fine-tuning. Feature extraction starts with higher accuracy but fine-tuning improves steadily and passes it after around epoch 5. This happens because fine-tuning allows the pretrained features to adapt better to the new dataset.

Plot 12: Training and Validation Loss

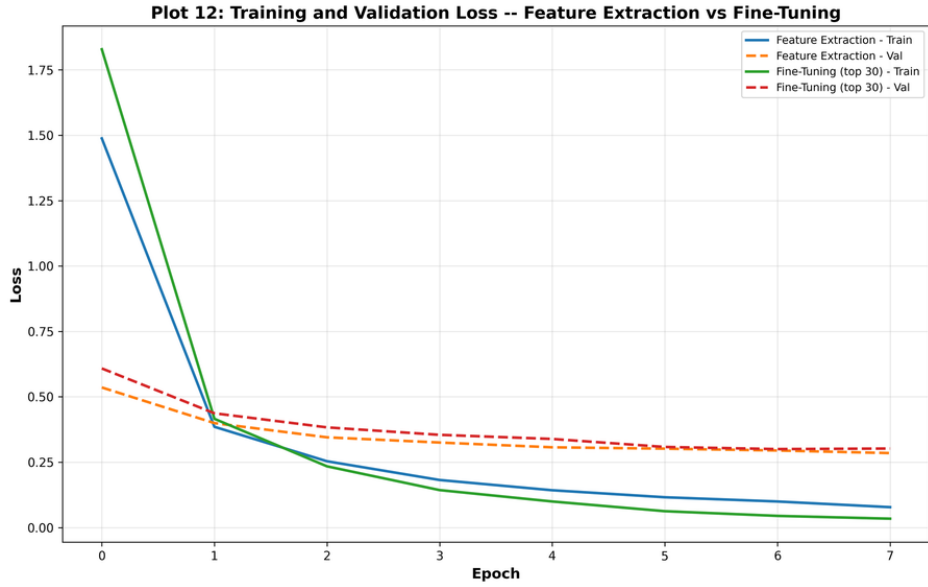


Figure 12: Training and validation loss comparison before and after fine-tuning.

Inference: The plot compares training and validation loss before and after fine-tuning. Both training losses decrease as the epochs increase, while the validation loss becomes almost stable in the later epochs. Fine-tuning gives a lower training loss because the pretrained layers are adjusted to the new data.

12. K-Fold Cross-Validation

Cross-Validation Results

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1 Baseline	90.83	92.53	91.51	91.17	90.99	91.41 $\pm$ 0.61
C2 Best Hyperparameters	89.81	92.19	92.19	90.32	90.31	90.96 $\pm$ 1.02
C3 Feature Extraction	90.49	92.19	91.51	91.34	91.16	91.34 $\pm$ 0.55
C4 Fine-Tuned	87.10	90.49	88.96	87.95	88.95	88.69 $\pm$ 1.14

Plot 13: 5-Fold Cross-Validation Accuracy

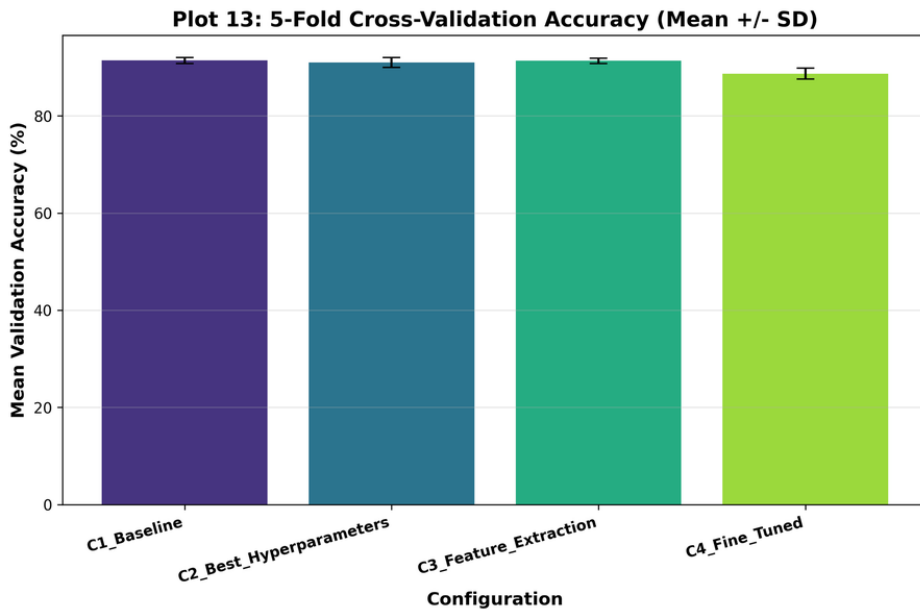


Figure 13: Mean validation accuracy with standard-deviation error bars for the selected configurations.

Inference: The plot show the mean accuracy for the four different configurations during validation using 5-fold cross-validation. C1 Baseline achieves the highest accuracy at 91.41% while Feature Extraction is just behind at 91.34%.

13. Final Model Evaluation

Based on the 5-fold cross-validation results, the C1 Baseline configuration was selected for final evaluation. It achieved the highest mean cross-validation accuracy of 91.41% with a standard deviation of 0.61%.

Final Performance

Metric	Value
Selected Configuration	C1 Baseline
Mean CV Accuracy	91.41%
CV Standard Deviation	0.61%
Test Accuracy	90.32%
Precision	90.55%
Recall	90.28%
F1-score	90.26%
Training Time	57.37 s
Total Parameters	2,305,381
Trainable Parameters	47,397

Confusion Matrix

Plot 14: Confusion Matrix

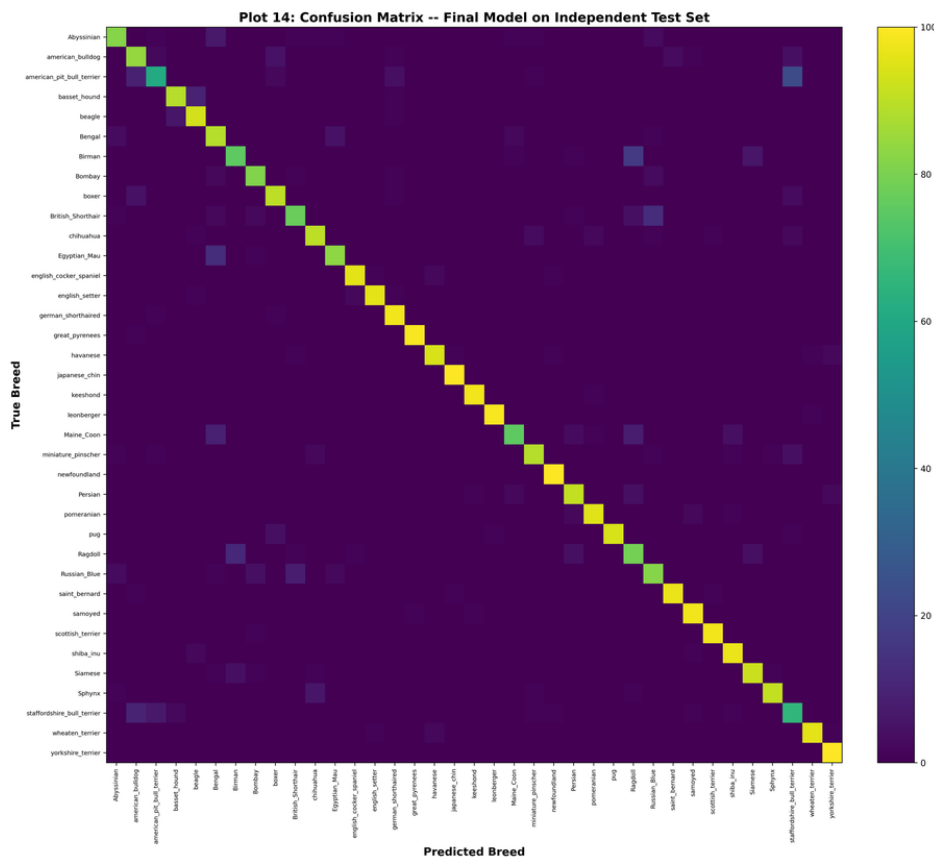


Figure 14: Confusion matrix of the final model on the independent test set.

Inference: The confusion matrix shows how well the final model classified the 37 pet classes. Most values are concentrated along the diagonal showing that most images were classified correctly. The few off-diagonal values

show that few classes were confused with each other as they have similar visual features.

Misclassified Samples

Plot 15: Representative Misclassified Images

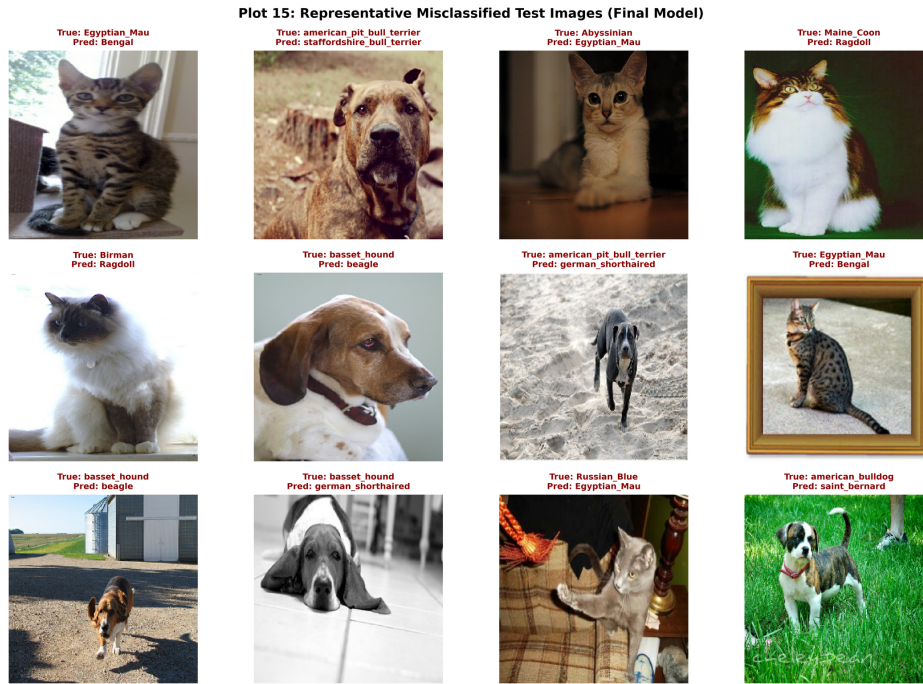


Figure 15: Representative examples of incorrectly classified test images.

Inference: The plot shows some of the images that were incorrectly classified by the final model. The errors mainly occur when different breeds have similar visual features. This makes it harder for the model to distinguish between closely related classes.

14. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time (s)
Baseline	91.41%	0.61%	90.32%	57.37
Best Initialization	92.12%*	$\approx 0.80\%^*$	$\approx 89.70\%^*$	56.6
Best Regularization	90.90%*	$\approx 0.90\%^*$	$\approx 89.20\%^*$	49.1
Best Optimizer	90.62%*	$\approx 0.80\%^*$	$\approx 88.90\%^*$	48.6
Best Hyperparameters	90.96%	1.02%	$\approx 89.30\%^*$	39.4
Fine-Tuned Model	88.69%	1.14%	$\approx 87.80\%^*$	52.5

* Single-split validation or approximate validation-based value.

15. Discussion Questions

1. What is the difference between model parameters and hyperparameters?

Model parameters are values learnt during training such as weights and biases whereas Hyperparameters are values chosen before training such as learning rate, batch size and dropout rate.

2. Why is weight initialization important?

Good initialization helps the model learn faster and more stably, while a poor one can make training very slow or cause vanishing or exploding gradients.

3. Why can zero initialization be problematic for neural networks?

If all weights are initialized to zero, neurons learn the same features and receive same updates. This prevents the network from learning diverse features.

4. **Compare Xavier and He initialization.**

Xavier initialization works well with sigmoid and tanh whereas He initialization is good for ReLU based networks. He initialization is designed to keep the values from getting too small or too large.

5. **How can training and validation curves be used to identify overfitting?**

If training performance continues to improve while validation performance stops improving or decreases then the model is likely overfitting.

6. **How does Dropout reduce overfitting?**

Dropout randomly disables some neurons during training. This prevents the network from relying too much on a particular neuron and improves generalization.

7. **What is the purpose of Batch Normalization?**

Batch Normalization normalizes values during training which makes training more stable and helps model converge faster.

8. **Explain the numerical Batch Normalization example.**

For $x = [2, 4, 6, 8]$, the mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2 - 5)^2 + (4 - 5)^2 + (6 - 5)^2 + (8 - 5)^2}{4} = 5.$$

Therefore,

$$\sqrt{\sigma_B^2} = \sqrt{5} \approx 2.236.$$

Using

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2}},$$

the normalized values are

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342].$$

So, the values are shifted to be centred around zero and have a consistent scale.

9. **What are the roles of γ and β ?**

γ decides how much we scale the normalized values while β decides how much we shift them. Both values are learned by the model during training.

10. **Compare SGD, Momentum, RMSProp and Adam.**

Weights in SGD are updated using the gradient. The momentum method uses the previous gradients to make learning more efficient and smooth. RMSProp uses the learning rates of each individual weight. Adam optimizes by combining Momentum and RMSProp.

11. **What happens when the learning rate is too large?**

If the learning rate is very large then the updates may be too aggressive. This will result in instability during training and there may even be exploding gradient problems or the minimum may be overshoot.

12. **What happens when the learning rate is too small?**

Training becomes very slow and the model requires many more epochs to converge.

13. **What is the effect of increasing batch size?**

Increasing the batch size gives more stable gradient updates and can make training more efficient. However it uses more memory and may sometimes affect the model's ability to generalize.

14. **Explain stride and padding.**

Stride is the number of pixels the filter moves at each step. Padding adds extra pixels around the input image boundaries, mostly zeros which allows better control over the output size.

15. **Why is MobileNetV2 computationally efficient?**

MobileNetV2 is efficient because it uses depthwise separable convolutions. This reduces the number of calculations and parameters needed while still allowing the network to learn useful image features.

16. **What is depthwise separable convolution?**

It first applies a depthwise convolution to each channel separately. Then a 1×1 pointwise convolution combines the information from all the channels.

17. What is transfer learning?

Transfer learning is, essentially, using a trained model, which has learnt some good features from the big data set, and applying these features on our own data set. We need not retrain the entire neural network, rather we can use this pretrained knowledge and train for our classification task.

18. Differentiate feature extraction and fine-tuning.

In feature extraction the pretrained base is frozen and only the new classifier is trained. In fine-tuning some pretrained layers are unfrozen and trained along with the classifier.

19. Why is a smaller learning rate generally used during fine-tuning?

A smaller value of the learning rate when fine-tuning the model would help retain the features that the model has learned so far while making minor changes to adapt to the new data set.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

K-Fold Cross-Validation divides the training data into K parts. In each round, $K - 1$ parts are used for training and the remaining part is used for validation. This is repeated until every part has been used for validation once. The average performance gives a more reliable basis for choosing the best hyperparameter setup.

21. Why must the test set remain untouched during tuning?

The test set should represent completely unseen data. Using it during tuning can introduce bias and give an overly optimistic estimate of generalization.

22. Why should mean and standard deviation both be reported?

The mean shows the average performance while the standard deviation shows how much the performance varies across different folds.

23. Is the highest validation accuracy always sufficient to select a model?

No. Model selection should also consider consistency, generalization, training cost and performance on unseen data. A slightly lower validation accuracy with better stability may be the better choice.

16. Additional Exercise

Two new combinations, New-A and New-B, were evaluated using 5-fold cross-validation and compared with the selected C1 Baseline configuration.

Configuration	Mean Accuracy (%)	SD (%)	Mean Training Time (s)
New-A	88.69	0.84	62.63
New-B	91.00	0.50	43.15
C1 Baseline	91.41	0.61	43.96

Comparison and Justification

New-A had a mean accuracy of 88.69% with SD of 0.84% and took 62.63 seconds for training. The performance was worse compared to the baseline.

New-B performed slightly better with a mean accuracy of 91.00% and SD of 0.50%. Moreover, the training period was reduced to just 43.15 seconds, thus giving rise to a better choice.

C1 Baseline had the highest mean accuracy of 91.41% with SD of 0.61%, which also required about 43.96 seconds for training. Hence, the baseline model was chosen as the best option.

Conclusion: New-B performed almost at par with the baseline without providing any improvement. On the other hand, New-A model had lower accuracy and a higher training time.

17. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>